

2교시: Log, 설정, 비밀값 - stdout/stderr, error message, env var, 비밀값 비노출

수업 목표

- log, 설정, 비밀값을 구분한다.
- 에러 메시지를 원인 분석 확인 기록으로 기록한다.
- 비밀값은 값이 아니라 존재와 관리 방식만 문서화한다.

오늘 반드시 가져갈 것

필수 개념	왜 필수인가	놓치면 생기는 문제	확인 기록
로그	실행 중 일어난 사건과 요청 결과를 보여준다.	실패 원인을 감으로 추측한다.	요청 로그, 에러 메시지 발체
설정	실행 방식을 바꾸는 값이며 문서화 대상이다.	포트나 경로가 달라져도 원인을 찾지 못한다.	PORT=8000, 설정 key 설명
비밀값	노출되면 피해가 생기는 token/password/key다.	README, 화면 공유, git에 credential이 남는다.	값 제외 메모, 관리 위치만 기록
발체 기준	필요한 부분만 남기고 민감정보는 가린다.	로그 전체 복사로 개인정보나 token이 섞인다.	안전한 로그 한 줄, 제거한 값 표시

챌린저 복구 기준

- PORT=8000은 공개해도 되는 설정 예시지만, token 문자열은 절대 적지 않는다.
- 로그를 복사하기 전 "이 줄을 공개 채팅에 붙여도 되는가"를 먼저 확인한다.
- 에러 메시지는 숨길 대상이 아니라 원인 분석 자료다. 다만 민감정보는 제거한다.

50분 흐름

Time	Activity
0-5분	local server 상태 코드 확인 기록 확인
5-15분	log, 설정, 비밀값 정의와 차이 설명
15-30분	env key와 request log 확인
30-40분	비밀값 노출 위험과 README 기록 기준 정리
40-50분	실행 조건 표로 연결

0-5분 local server 상태 코드 확인 기록 확인

- 진행: local server 상태 코드 확인 기록 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

상세 설명

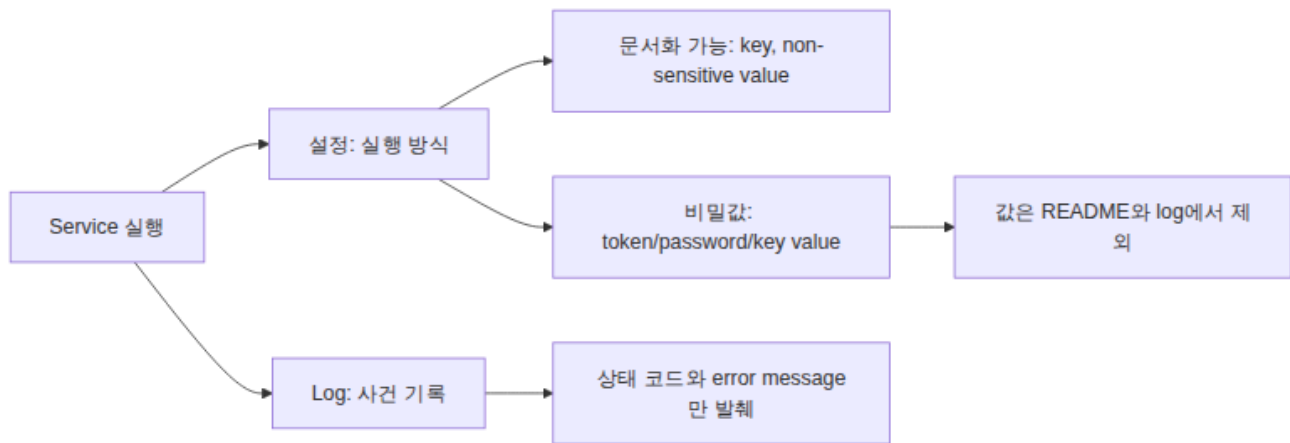
Log는 실행 중인 system이 남기는 사건 기록이다. 설정은 실행 방식을 바꾸는 값이다. 비밀값은 보호해야 하는 credential, token, password, private key 같은 값이다. 세 가지는 운영에서 자주 함께 보이지만 다르게 다뤄야 한다.

좋은 README는 설정 key 이름과 기본값은 설명하지만 비밀값 값은 쓰지 않는다. 예를 들어 PORT=8000은 기록할 수 있지만, API token 값은 기록하면 안 된다. log도 마찬가지로 error message와 상태 코드는 확인 기록이 될 수 있지만 credential이 섞였는지 확인해야 한다.

Visual 1: Log, 설정, 비밀값 경계



이 이미지는 log, 설정, 비밀값을 모두 “텍스트”로 보지 않고 운영상 다른 보호 수준을 갖는 자료로 구분하게 한다. 강사는 비밀값 값 자체가 아니라 비밀값을 제외하는 판단 기준만 다룬다.



학생 기록에는 비밀값 값이 들어가면 안 된다. "있다/필요하다/어디서 관리한다"는 기록하고, 실제 값은 쓰지 않는다.

5-15분 log, 설정, 비밀값 정의와 차이 설명

- 진행: log, 설정, 비밀값 정의와 차이 설명
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

핵심 구분

항목	예	README에 적어도 되는가
Log	GET / HTTP/1.1" 200	일부 가능
설정 key	PORT	가능
설정 값	PORT=8000	민감하지 않으면 가능
비밀값 key name	GITHUB_TOKEN	필요 시 가능
비밀값 값	실제 token 문자열	불가
Error message	File not found	가능

Visual 2: 캡처 가이드

캡처 대상	기록 가능한 예	보호할 예
설정 key	PORT	없음
설정 값	PORT=8000	private endpoint 전체
비밀값 key name	GITHUB_TOKEN	token value
log	GET /no-such-file.html 404	credential이 포함된 전체 log
error	File not found	사용자 개인정보가 섞인 stack trace

캡처 전에는 한 번 멈추고 "이 줄을 친구에게 보여줘도 안전한가"를 확인한다. 안전하지 않으면 값을 가리고 원인 분석에 필요한 부분만 남긴다.

15-30분 env key와 request log 확인

- 진행: env key와 request log 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

명령 절차

```
env | grep -E 'SHELL|HOME|PATH'
```

서버 terminal에서 request log를 관찰한다.

```
GET / HTTP/1.1 GET /index.html HTTP/1.1
```

실패 log를 만들기 위해 새 terminal에서 없는 파일을 요청한다.

```
curl -I http://localhost:8000/no-such-file.html
```

확인 질문

- PORT=8000은 설정인가 비밀값인가?
- token 값을 README에 쓰면 어떤 위험이 생기는가?
- request log에서 symptom을 찾는 방법은 무엇인가?

30-40분 비밀값 노출 위험과 README 기록 기준 정리

- 진행: 비밀값 노출 위험과 README 기록 기준 정리
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

다음 주차 매핑

Docker는 -e와 env file, Kubernetes는 Secret/ConfigMap, AWS는 Parameter Store/Secrets Manager, Terraform은 sensitive variable로 이 구분을 다룬다.

예상 결과

- env | grep ...은 일반 환경 key/value 일부를 보여준다.
- 없는 파일 요청은 보통 404 상태 코드를 만든다.
- 서버 terminal에는 요청 경로와 상태 코드가 log로 남는다.

흔한 오해

오해	교정
설정과 비밀값은 모두 환경변수라서 같다.	비밀값은 노출되면 피해가 생기는 값이다. 관리 기준이 다르다.
log는 전부 복사해도 된다.	log에 token, email, 경로, 개인정보가 섞일 수 있다. 필요한 부분만 발췌한다.
에러 메시지는 실패라 숨겨야 한다.	에러 메시지는 RCA의 핵심 확인 기록이다. 민감정보만 제거한다.

40-50분 실행 조건 표로 연결

- 진행: 실행 조건 표로 연결
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

실습 확인 기록

확인 항목	값
설정 key list	
request log example	
404 log example	
비밀값 비노출 note	

학술 근거와 DevOps insight

Twelve-Factor App은 설정을 코드와 분리하라고 설명한다. 현업에서는 비밀값 노출이 보안 사고로 바로 이어질 수 있으므로 "설정 이름은 문서화하되 값은 안전한 저장소에 둔다"는 원칙을 초반부터 익혀야 한다.

평가 기준

기준	2점 확인 기록
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~5 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.

공식/학술 근거 링크

- OWASP Secrets Management Cheat Sheet,
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html - configuration과 비밀값을 구분해야 하는 보안 기준이다.

- GitHub Secret Scanning, <https://docs.github.com/en/code-security/secret-scanning/enabling-secret-scanning-features> - 저장소에 비밀값이 노출될 때의 탐지와 보호 기준이다.
- NIST AI Risk Management Framework, <https://www.nist.gov/itl/ai-risk-management-framework> - AI 도움을 받을 때도 정보 노출과 권한 위험을 점검해야 하는 기준이다.